Lesson9_R_DataFrame

Paul Lott

2024-05-13

R Dataframes

Data Frames are similar to Python Pandas Dataframes, where each column can contain a different type of data. R Data Frames are a mix between a matrix and a list. They are one of the fundamental data structures used by most R software.

Creating Data Frames

To create a data frame, you'll use `data.frame()` constructor.

Usage:

```
data.frame(..., row.names = NULL, check.rows = FALSE,
           check.names = TRUE, fix.empty.names = TRUE,
           stringsAsFactors = FALSE)
```

`...` in data.frame is either a value/vector or a tagging name and value/vector. If a tag name is used, the tag name will be used as the column name. If tag names are not used, then the columns are named will be taken from the first element of vector or value.

```r
df = data.frame(FirstName=c("John", "Travis", "Reese"),
                LastNames=c("Hurst", "Villegas", "Moran"),
                Age=c(21, 35, 48),
                Include=c(TRUE, FALSE, TRUE),
                Date=c("2023-02-22", "2022-04-13", "2024-1-2")
                )
df
```

```
##   FirstName LastNames Age Include       Date
## 1      John     Hurst  21    TRUE 2023-02-22
## 2    Travis  Villegas  35   FALSE 2022-04-13
## 3     Reese     Moran  48    TRUE   2024-1-2
```

Reading & Writing a data.frame or matrix

You can write a data.frame or matrix using the `write.table()` function and read them using `read.table()`

Syntax:

```
x = read.table(file, header=<TRUE/FALSE>, sep="",...)
x = read.csv(file, header=<TRUE/FALSE>, sep=",", ...)
x = read.csv2(file, header=<TRUE/FALSE>, sep=";", ...)
x = read.delim(file, header=<TRUE/FALSE>, sep="\t", ...)

write.table(x, file="", sep=" ", row.names=<TRUE/FALSE>, col.names=<TRUE/FALSE>, ...)
```

```
write.csv(...)
write.csv2(...)
```

```r
x = read.table("https://raw.githubusercontent.com/mwaskom/seaborn-data/master/penguins.csv", header=TRU
head(x)
```

```
##   species    island bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
## 1  Adelie Torgersen           39.1          18.7               181        3750
## 2  Adelie Torgersen           39.5          17.4               186        3800
## 3  Adelie Torgersen           40.3          18.0               195        3250
## 4  Adelie Torgersen             NA            NA                NA          NA
## 5  Adelie Torgersen           36.7          19.3               193        3450
## 6  Adelie Torgersen           39.3          20.6               190        3650
##      sex
## 1   MALE
## 2 FEMALE
## 3 FEMALE
## 4
## 5 FEMALE
## 6   MALE
```

```r
write.csv(x, file="Penguins.csv", row.names=FALSE)
```

Summarize Dataframe

`summary()` gives a summary of the dataframe, similar to Pandas .describe() function. If the columns values are numeric, it'll provide the Min, Max, Quartiles, Mean and Median values.

```r
summary(x)
```

```
##    species             island          bill_length_mm  bill_depth_mm
##  Length:344         Length:344         Min.   :32.10   Min.   :13.10
##  Class :character   Class :character   1st Qu.:39.23   1st Qu.:15.60
##  Mode  :character   Mode  :character   Median :44.45   Median :17.30
##                                        Mean   :43.92   Mean   :17.15
##                                        3rd Qu.:48.50   3rd Qu.:18.70
##                                        Max.   :59.60   Max.   :21.50
##                                        NA's   :2       NA's   :2
##  flipper_length_mm  body_mass_g       sex
##  Min.   :172.0     Min.   :2700   Length:344
##  1st Qu.:190.0     1st Qu.:3550   Class :character
##  Median :197.0     Median :4050   Mode  :character
##  Mean   :200.9     Mean   :4202
##  3rd Qu.:213.0     3rd Qu.:4750
##  Max.   :231.0     Max.   :6300
##  NA's   :2         NA's   :2
```

Accessing Items

You can access items in the data.frame by using either:

1. **Single Brackets `[]`**
   - Single Columns by Indices

   ```r
   print(df[1])
   ```

   ```
   ##   FirstName
   ```

```
## 1      John
## 2    Travis
## 3     Reese
```

- Element by Row and Columns

```
print(df[1,2])
```

```
## [1] "Hurst"
```

- Column by indices

```
print(df[,1])
```

```
## [1] "John"   "Travis" "Reese"
```

- Row by Indices

```
df[1,]
```

```
##   FirstName LastNames Age Include       Date
## 1      John     Hurst  21    TRUE 2023-02-22
```

2. **Double Brackets `[[]]` :** Access the corresponding column by name

```
df[['Age']]
```

```
## [1] 21 35 48
```

3. **Dollar Sign: `$`** To access column by columns Name

```
print(df$Age)
```

```
## [1] 21 35 48
```

Adding a Column or Columns

Recall using Matrices, we could use `cbind()` to add new columns to the matrix. However, that column had to be the same datatype as the matrix. With data.frames, we can also add columns using `cbind()` but that column can be any R Datatype.

Let's add Case/Controls status to the data.frame as a Factor. We'll first create a vector that corresponds to each row and then create a factor from the vector.

Finally, we'll use `cbind()` to add our new columns called `Case_Control` to the `df` and assign the new data.frame to a variable called `new_df`

```
df
```

```
##   FirstName LastNames Age Include       Date
## 1      John     Hurst  21    TRUE 2023-02-22
## 2    Travis  Villegas  35   FALSE 2022-04-13
## 3     Reese     Moran  48    TRUE   2024-1-2
```

```
ccs = c("Case", "Control", "Control")
ccs_f = factor(ccs)

new_df = cbind(df, Case_Control=ccs_f)
print(new_df)
```

```
##   FirstName LastNames Age Include       Date Case_Control
## 1      John     Hurst  21    TRUE 2023-02-22         Case
## 2    Travis  Villegas  35   FALSE 2022-04-13      Control
```

```
## 3     Reese     Moran  48    TRUE   2024-1-2      Control
```

If we're adding multiple columns to the dataset, we can also use `cbind()` similar to how we did with Matrices. But we need to ensure that the amount of rows are equal.

Adding a Row or Rows

To add a row or rows to a data.frame, we use `rbind()` . We can add either a single row or another data.frame. If we're adding rows, we need to make certain that the number of elements in the new rows match our given dataframe.

In-class Exercise:

Run the two exercise code blocks below. What are the differences between the two different. Explain the difference between the two resulting data.frame. If you can't spot any differences, use `summary()` to check for additional differences.

Exercise Block 1:

```r
new_entry = list("John", "Anderson", 48, TRUE, "2024-5-20", "Case")

newer_df1 = rbind(new_df, new_entry)
print(newer_df1)
```

```
##   FirstName LastNames Age Include       Date Case_Control
## 1      John     Hurst  21    TRUE 2023-02-22         Case
## 2    Travis  Villegas  35   FALSE 2022-04-13      Control
## 3     Reese     Moran  48    TRUE   2024-1-2      Control
## 4      John  Anderson  48    TRUE  2024-5-20         Case
```

```r
summary(newer_df1)
```

```
##   FirstName          LastNames              Age        Include
##  Length:4           Length:4           Min.   :21.0   Mode :logical
##  Class :character   Class :character   1st Qu.:31.5   FALSE:1
##  Mode  :character   Mode  :character   Median :41.5   TRUE :3
##                                        Mean   :38.0
##                                        3rd Qu.:48.0
##                                        Max.   :48.0
##      Date            Case_Control
##  Length:4           Case   :2
##  Class :character   Control:2
##  Mode  :character
##
##
##
```

Exercise Block 2:

```r
new_entry = c("John", "Anderson", 48, TRUE, "2024-5-20", "Case")

newer_df2 = rbind(new_df, new_entry)
print(newer_df2)
```

```
##   FirstName LastNames Age Include       Date Case_Control
## 1      John     Hurst  21    TRUE 2023-02-22         Case
## 2    Travis  Villegas  35   FALSE 2022-04-13      Control
```

```
## 3     Reese     Moran  48    TRUE   2024-1-2      Control
## 4      John  Anderson  48    TRUE  2024-5-20         Case
```

```r
summary(newer_df2)
```

```
##   FirstName          LastNames             Age              Include
##  Length:4           Length:4           Length:4           Length:4
##  Class :character   Class :character   Class :character   Class :character
##  Mode  :character   Mode  :character   Mode  :character   Mode  :character
##      Date            Case_Control
##  Length:4           Case   :2
##  Class :character   Control:2
##  Mode  :character
```

If the amount of elements doesn't match the number of columns. It will be trimmed to match. (Note: the result is missing the value 1899)

```r
new_entry = list("John", "Anderson", 48, TRUE, "2024-5-20", "Case", 1899)
unequal_cols = rbind(new_df, new_entry)
print(unequal_cols)
```

```
##   FirstName LastNames Age Include       Date Case_Control
## 1      John     Hurst  21    TRUE 2023-02-22         Case
## 2    Travis  Villegas  35   FALSE 2022-04-13      Control
## 3     Reese     Moran  48    TRUE   2024-1-2      Control
## 4      John  Anderson  48    TRUE  2024-5-20         Case
```

If you use a Named List, whose names match the column names then order of items doesn't matter.

```r
new_entry = list( LastNames="Anderson", Age=48, Date="2024-5-20", Case_Control="Case", FirstName="John"
equal_cols = rbind(new_df, new_entry)
print(equal_cols)
```

```
##   FirstName LastNames Age Include       Date Case_Control
## 1      John     Hurst  21    TRUE 2023-02-22         Case
## 2    Travis  Villegas  35   FALSE 2022-04-13      Control
## 3     Reese     Moran  48    TRUE   2024-1-2      Control
## 4      John  Anderson  48    TRUE  2024-5-20         Case
```

However, if you try to include a *name* that isn't in the table, you'll get an error.

```r
new_entry = list( LastNames="Anderson", Age=48, Date="2024-5-20", Case_Control="Case", FirstName="John"
unequal_cols = rbind(new_df, new_entry)
```

```
## Error in rbind(deparse.level, ...): numbers of columns of arguments do not match
```

```r
print(unequal_cols)
```

```
##   FirstName LastNames Age Include       Date Case_Control
## 1      John     Hurst  21    TRUE 2023-02-22         Case
## 2    Travis  Villegas  35   FALSE 2022-04-13      Control
## 3     Reese     Moran  48    TRUE   2024-1-2      Control
## 4      John  Anderson  48    TRUE  2024-5-20         Case
```

Subsetting or Filtering your Data.Frame

Reference: R Keep or Drop Columns

You can subset data.frames using the brackets `[]` and instead of just supplying an single **indice/name** - supply a vector of **indices** or **names**

```
df[c(1,3), c("FirstName", "Age")]
```

```
##   FirstName Age
## 1      John  21
## 3     Reese  48
```

You can also use comparison operators similar to subsetting vectors to choose rows that meet a specific criteria. `which()` function takes applies a single or multiple comparisons and returns the indices where the comparison is TRUE.

For which `df` rows are the `'Age'>22`

```
which(df$'Age'>22)
```

```
## [1] 2 3
```

Then we just need to plug that into the where we access rows in the `[]` operator and we get the subsetted table filtered on `Age>22`.

```
df[which(df$'Age'>22),]
```

```
##   FirstName LastNames Age Include       Date
## 2    Travis  Villegas  35   FALSE 2022-04-13
## 3     Reese     Moran  48    TRUE   2024-1-2
```

Another methods is `subset(x, comparison, optional(select=c(column names))`

Notice that in subset, we don't need to wrap the column names in quotes.

```
subset(df, Age>22 & Include, select=c(FirstName, LastNames))
```

```
##   FirstName LastNames
## 3     Reese     Moran
```

Removing Rows or Columns

If you need to remove Rows or Columns, you just need to pass a negative indice that corresponds to that row or column.

```
df[-1,]
```

```
##   FirstName LastNames Age Include       Date
## 2    Travis  Villegas  35   FALSE 2022-04-13
## 3     Reese     Moran  48    TRUE   2024-1-2
```

```
df[,-2]
```

```
##   FirstName Age Include       Date
## 1      John  21    TRUE 2023-02-22
## 2    Travis  35   FALSE 2022-04-13
## 3     Reese  48    TRUE   2024-1-2
```

```
df[-which(df$'Age'>22),]
```

```
##   FirstName LastNames Age Include       Date
## 1      John     Hurst  21    TRUE 2023-02-22
```

If you are using the `subset()` function, you can just make the `select` vector negative (ie 'select=-c(FirstName, Include)) would exclude FirstName and Include from the data.frame.

```r
subset(df, Age>22 & Include, select=-c(FirstName, Include))
```

```
##   LastNames Age     Date
## 3     Moran  48 2024-1-2
```

What about Date? It's still a character type.

```r
df
```

```
##   FirstName LastNames Age Include       Date
## 1      John     Hurst  21    TRUE 2023-02-22
## 2    Travis  Villegas  35   FALSE 2022-04-13
## 3     Reese     Moran  48    TRUE   2024-1-2
```

R does have a `Date` object, and we can convert it to dates using `as.date()` either when we create the data.frame or after.

```r
df[['Date']] = as.Date(df[['Date']])
df
```

```
##   FirstName LastNames Age Include       Date
## 1      John     Hurst  21    TRUE 2023-02-22
## 2    Travis  Villegas  35   FALSE 2022-04-13
## 3     Reese     Moran  48    TRUE 2024-01-02
```

```r
c = c(1,2,3)
is.vector(c)
```

```
## [1] TRUE
```

```r
as.character(c)
```

```
## [1] "1" "2" "3"
```

For more information about Dates: https://www.stat.berkeley.edu/~s133/dates.html

Now we've seen the major players in R datatypes and Objects. Let's step aside an look at conditional, loops, plotting.

We're going to then jump back in to exploring the **Tidyverse**. The Tidyverse is a group of tools built for data science that address some of R's peculiarities to make data analysis simpler and more powerful.

Exercise 1: Penguins Dataset

Import the Palmer Penguins dataset from above. You'll notice that the Species, Island, and Sex are all categorical. Convert Species, Island, and Sex columns to Categorical `factor`

Get the summary information (mean, median, max, min, quartile) for each species.